

Reliable and Scalable Product

This project is a comprehensive Site Reliability Engineering (SRE) challenge focused on the lifecycle of a cloud-native application. Rather than prioritizing feature-heavy software development, the project emphasizes "Operations First"—building a resilient, observable, and automated infrastructure using Kubernetes as the foundation.

I M0: Product Pitch & Operational Planning

Objectives

Define the Product Domain by proposing a clear, cohesive product idea within a specific domain or industry (e.g., e-commerce cart, IoT data ingestion, ticketing system).

Establish the Tech Stack, by selecting a multi-tier technical stack (e.g., frontend, backend, database/cache) and justify these choices from an operational perspective, rather than just a development one.

Prioritize Operations over Development. Demonstrate a plan to keep application development simple and feature-light, ensuring the team's primary effort is spent on Kubernetes deployment, resilience, and observability.

Identify Operational Challenges. Anticipate the architectural and operational complexities of running this specific product in a production environment.

Presentation & Planning Guidelines

To pass this milestone, your team must present your product vision and operational roadmap, covering the following key areas:

Product Vision & Use Case

Briefly describe what the product does and the specific area/industry it serves. Identify the 1-2 critical paths a user will take (e.g., "A user logs in and queries their account balance"). These are the journeys you will eventually monitor and protect with SLOs.

Architecture & Technical Stack

Present a high-level diagram of your proposed system. To be suitable for an SRE project, it must include at least a stateless compute tier (e.g., web server/API) and a stateful tier (e.g., a database, message queue, or cache).

You are free to choose any programming languages, frameworks, or databases. Explain why you chose this stack. (e.g., "We chose Go for the backend because of its low memory footprint in containers," or "We chose Redis because it is easy to cluster").

The "Operations First" Strategy

How will you ensure your team does not get bogged down in software development? As an example by relying on simple REST APIs, using mock data generators, skipping complex UI/UX design, or utilizing existing open-source microservice templates. Outline

how the team will organize to focus on SRE tasks (infrastructure as code, CI/CD, monitoring) alongside basic application scaffolding.

Operational Foresight (Anticipating Pain Points)

- What are the critical dependencies of your system?
- What component do you expect to be the most difficult to scale, deploy, or recover in the event of a failure?
- Can you do updates to each component?

I M1: Baseline Kubernetes Deployment

Objectives

Establish a Baseline and deploy the core product onto a basic Kubernetes cluster (e.g. using `k3d`), ensuring all necessary microservices or components are communicating correctly. Then, validate functionality by demonstrating that the core user journeys (CUJs) operate successfully from end to end.

Establish reproducibility and move away from manual configurations by defining the initial infrastructure and deployment as code (e.g., YAML manifests).

Assess the architectural you envisioned. Use this initial deployment as a proof-of-concept to evaluate whether the current operational strategy is viable for future scaling and reliability requirements.

Report Requirements

To pass this milestone, your team must submit a report covering the following areas.

Note: This phase is about functional verification (smoke testing), not performance benchmarking. We just need to prove the system works as intended.

Architecture & Stack Operation

Component Mapping the product by describing the product's architecture and how it maps to Kubernetes resources (e.g., Deployments, Pods, Services, Ingress, Persistent Volumes). Provide a justification for your decisions.

Identify the main data flow and briefly explain how a request travels through your stack components.

Deployment Strategy & Configuration

Detail how the application is deployed (e.g., standard manifests, introduction of Helm/Kustomize), the overall architecture and the reason for it.

Explain how environment variables, secrets, and configurations are handled (e.g., ConfigMaps, Secrets), stored and passed into software.

Justify why you chose this deployment method. Describe at least one alternative option you considered, why you discarded it for now, and whether you might adopt it in a later milestone.

System Evaluation & Functional Verification

Explain how you verify the system is working. Did you implement basic Kubernetes Health Checks (Liveness/Readiness probes)? What is your definition of working?

How do you currently know what the system is doing? Even if it is just checking `kubectl logs` or basic metrics, document your current visibility. Identify what you can get and what is missing.

Based on your current architecture and deployment strategy, identify potential structural bottlenecks or single points of failure. What will break first when we start applying load in future milestones?

Deployment Environment:

Use a local environment on your laptop potentially inside a local virtual machine. We recommend Virtualbox, but if you are using ARM silicon, other options may be better suited.

- `k3d` or `kind` (Kubernetes in Docker): These are highly recommended for SREs because they allow you to easily spin up multi-node virtual clusters locally using Docker containers as the "nodes."
- `Minikube` / `Docker Desktop`: Good for single-node testing, but less ideal for practicing distributed system concepts.

M2: Resilience, Automation, and Observability

Objectives

Demonstrate that the system is highly available (HA) and can seamlessly route traffic around component, pod, or node failures without significant service degradation.

Demonstrate the support and understanding of product Observability. Transition from basic "is it running?" checks to deep system visibility, ensuring the team can proactively identify issues and understand system health over time.

Eliminate operational toil by implementing automation for repetitive tasks, applied for both the application and the infrastructure.

Critically assess the decisions made in Milestone 1, detailing how the architecture has evolved to meet the demands of automated scaling and resilience.

Report Requirements

At this milestone, the team must submit a comprehensive report, with screenshots (Evidences), critical analysis, and source code, detailing the robust operation of the cluster, covering the following areas:

Automation & Deployment Pipeline

Describe the automated deployment pipeline. How does code go from an intent to a running Pod in the cluster? (deployment triggered by a script. We will not aim at CI/CD but a at single script)

When deploying services, describe how configuration and secret management are handled. Specifically, how are updates and rollbacks handled automatically and these items included in the process.

Resilience, Scaling, & Recovery

Detail how the application is configured to survive failures of some components. Explain your strategies for redundancy (e.g., replica sets, pod anti-affinity, multi-zone deployments if applicable).

Describe the auto-scaling strategies implemented. Are you using the Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA), or Cluster Autoscaler? Under what conditions does the system scale? Can you demonstrate that it works?

Disaster Recovery (DR) & State Management is also relevant. Explain how the system handles state, for storage and databases. If a database or persistent volume fails, how is data recovered? Detail your backup and restoration strategies.

Observability & Monitoring

Define the core Service Level Indicators (e.g., latency, error rate) and your target Service Level Objectives for the product. Consider the most relevant user Journeys.

Describe your monitoring stack (e.g., Prometheus, Grafana, OpenTelemetry). How are metrics, logs, and traces aggregated, and how can they be used to better know the product?

Demonstrate how you monitor long-term trends (e.g., resource saturation, memory leaks, cyberattacks). Describe the alerting rules you have configured to notify the team before an issue impacts the user.

Architectural Review & Assessment

This is also an opportunity to review your assumption and decisions. What assumptions from the first deployment were incorrect? What configurations had to be changed to support resilience and automation? Also, what are the known limits of your current highly-available setup?

Deployment Environment:

- Use the local environment that you used in the previous milestone. Grade will be limited to 16 points.
- Use the Department Cluster environment. Grade will not be limited.

Note: Because the system is shared, failures will occur, resource starvation will be real and optimizations may be needed!

Useful references

- `k3d` : <https://k3d.io/stable/>
- `k3s` : <https://k3s.io/>
- `minikube` : <https://minikube.sigs.k8s.io/docs/>
- `Awesome SRE` : <https://github.com/SquadcastHub/awesome-sre-tools>